

To search and list values, type:

```
Magazine = Query()
db.search(Magazine.type == 'OSFY')
[{'count': 700, 'type': 'OSFY'}]

db.search(Magazine.count > 750)
[{'count': 800, 'type': 'EFY'}]
```

For updating the values, use the following commands:

```
db.update({'count': 1000}, Magazine.type == 'OSFY')
db.all()

[{'count': 1000, 'type': 'OSFY'}, {'count': 800, 'type': 'EFY'}]
```

To remove values, type:

```
db.remove(Magazine.count < 900)
db.all()
[{'count': 800, 'type': 'EFY'}]
```

To delete all the values, give the following commands:

```
db.purge()
db.all()
[]
```

TinyDB enables developers to handle data in two different ways, as listed below:

- JSON
- In-memory

The default value is JSON, and if you want to change it to in-memory, you need to specify that explicitly.

```
from tinydb.storages import MemoryStorage
db = TinyDB(storage=MemoryStorage)
```

The detailed documentation on TinyDB API is available at <https://tinydb.readthedocs.io/en/latest/api.html>.

ZODB

ZODB is a native object database for Python. Its major features are:

- Seamless integration between code and database.
 - No separate language is needed for operations related to the database.
 - No database mapper is required.
- ZODB is better suited for the following scenarios:
- When the developer wants to focus more on the

application rather than building lots of database code.

- When the application has lots of complex relationships and data structures.
- When the data read operations are comparatively larger than the write operations.

At the same time, ZODB is not suitable for scenarios in which the application requires a high volume of data write operations.

To install ZODB, use the following command:

```
$ pip install ZODB
```

A simple code snippet to establish connection with a database is shown below:

```
import ZODB, ZODB.FileStorage
storage = ZODB.FileStorage.FileStorage('mydata.fs')
db = ZODB.DB(storage)
connection = db.open()
root = connection.root
```

ZODB allows developers to use a wide variety of storage options, as listed below:

- In-memory databases
- Local files
- Databases on remote servers
- Advanced options such as compressed and encrypted storage

The members of the object can be directly stored using ZODB functions. A detailed tutorial for using ZODB is available at <http://www.zodb.org/en/latest/tutorial.html>. ZODB has support for transactions also.

This article has attempted to provide an insight into the world of Python databases. There are many other databases apart from the three narrated in this article, such as buzbug (<http://buzbug.sourceforge.net/>) and CodernityDB (<http://labs.codernity.com/codernitydb/>). The bottomline is that all these database tools provide a Pythonic ambience when you work with them. 

References

- [1] <http://www.zodb.org>
- [2] <https://tinydb.readthedocs.io>
- [3] <https://pythonhosted.org/pickleDB/commands.html>

By: Dr K.S. Kuppusamy

The author is assistant professor of computer science at the School of Engineering and Technology, Pondicherry. He has 11+ years of teaching and research experience in academia and industry. He can be reached via mail at kskupu@gmail.com.